

Air University

Department of Computer Science

Introduction to Parallel & Distributed Computing (CS426L)

Lab Report 02: Hardware Topology, Threads, and Flynn's Taxonomy

Student Name	Muhammad Fahad Noor
Registration No.	232448
Section	BSCS-ev-7-C
Instructor	Muhammad Nauman Zafar Hashmi
Date	Wednesday, September 16, 2026

Section 1: Environment Setup

The .NET SDK was installed and verified using `dotnet --version`. A console project (CoreThreads) was created directly on Windows using Visual Studio Code and the C# Dev Kit. Task 1 of this lab the `lscpu / hwloc-ls` hardware-topology inspection, which requires a Linux environment via WSL was omitted from this submission with the instructor's permission; no WSL or Linux terminal was used at any point in completing Tasks 2 through 4.

```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\Lab\Lab2_PDC>dotnet --version
10.0.401
```

Section 2: Task 2 - One Thread Per Detected Core

Source file: *Program.cs*

```
using System;
using System.Threading;

class Program
{
    static void Worker(object? arg)
    {
        long id = (long)arg!;
        int cpu = Thread.GetCurrentProcessorId(); // snapshot only, OS may migrate the
thread
        Console.WriteLine($"Thread {id}: running on logical CPU {cpu}");
    }

    static void Main()
    {
        int numCores = Environment.ProcessorCount;
        Console.WriteLine($"Detected logical cores: {numCores}");

        // Dynamically sized never hard-code the thread count
        Thread[] threads = new Thread[numCores];

        for (int i = 0; i < numCores; i++)
        {
            int idx = i; // capture a local copy for this thread
            threads[i] = new Thread(() => Worker((long)idx));
            threads[i].Start();
        }

        for (int i = 0; i < numCores; i++)
        {
            threads[i].Join();
        }

        Console.WriteLine($"All {numCores} threads completed.");
    }
}
```

Results

Run	Detected Cores	Threads Created	All Threads Completed? (Y/N)
1	8	8	Y
2	8	8	Y

Run 1 Console Output

```
Problems 2 Output Debug Console Terminal Ports
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\Lab\Lab2_PDC>dotnet run
Detected logical cores: 8
Thread 4: running on logical CPU 5
Thread 3: running on logical CPU 3
Thread 5: running on logical CPU 0
Thread 1: running on logical CPU 7
Thread 2: running on logical CPU 0
Thread 0: running on logical CPU 4
Thread 6: running on logical CPU 4
Thread 7: running on logical CPU 6
All 8 threads completed.
```

Run 2 Console Output

```
Problems 2 Output Debug Console Terminal Ports
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\Lab\Lab2_PDC>dotnet run
Detected logical cores: 8
Thread 0: running on logical CPU 6
Thread 5: running on logical CPU 3
Thread 1: running on logical CPU 7
Thread 2: running on logical CPU 4
Thread 4: running on logical CPU 0
Thread 6: running on logical CPU 2
Thread 3: running on logical CPU 3
Thread 7: running on logical CPU 3
All 8 threads completed.
```

Observation Questions

1. Did the program create exactly the number of threads reported by Environment.ProcessorCount?

Yes. Because the thread array is sized to numCores = Environment.ProcessorCount and the creation loop runs exactly numCores times, the number of threads created always equals the

detected logical core count by construction the figures recorded in the Results table above for both runs confirm this.

2. Did every thread run on a distinct logical core, or did the operating system schedule more than one thread onto the same core at some point?

Not every thread ran on a distinct core. In the Run 1 output above, Thread 2 and Thread 5 both reported logical CPU 0, and Thread 0 and Thread 6 both reported logical CPU 4, while the remaining four threads each landed on a different core (3, 5, 6, and 7). Because Thread.GetCurrentProcessorId() only captures a single snapshot at the moment each thread prints its message, this kind of overlap is expected: the OS scheduler is free to place more than one thread on the same logical core at a given instant, or to migrate a thread mid-execution, and this reflects normal scheduler behaviour rather than an error in the program.

3. What would happen if significantly more threads than the number of available logical cores were created? Would the program still function correctly?

The program would still function correctly, since Thread.Start() and Thread.Join() do not require a one-to-one mapping between threads and physical hardware. The OS scheduler would time-slice the excess threads across the available logical cores, so total throughput would no longer scale linearly with thread count and context-switching overhead would increase, but no race conditions, deadlocks, or functional errors would result from over-subscribing the cores.

Section 3: Task 3 - Classifying the Machine Using Flynn's Taxonomy

Flynn's taxonomy classifies computer architectures along two independent dimensions: the number of concurrent instruction streams and the number of concurrent data streams.

Category	Instruction Streams	Data Streams	Typical Example
SISD	Single	Single	Classic single-core, non-vectorized execution
SIMD	Single	Multiple	SSE/AVX CPU extensions, GPU cores
MISD	Multiple	Single	Rare; fault-tolerant redundant systems
MIMD	Multiple	Multiple	Multicore CPUs, clusters, distributed systems

Evidence and Classification

With Task 1 omitted, the only hardware evidence available for this classification is what Task 2 itself produced.

Evidence	Observation
Logical core count (Environment.ProcessorCount, Task 2)	8

Independent instruction streams observed (Task 2 threads)	8
Primary Flynn classification for the machine as a whole	MIMD
Flynn classification of a single core in isolation	SISD at the level of a single scalar instruction stream

The lab machine is classified as MIMD at the machine level: Task 2 detected multiple logical cores and demonstrated multiple independent threads, one per core, each executing the same compiled Worker method but as an independent instruction stream operating on independent data (its own captured index and its own snapshot from Thread.GetCurrentProcessorId()). Independent cores executing independent instruction streams on independent data is exactly the MIMD definition, and Task 2's measured core count and thread count are the concrete evidence supporting it.

Considered in isolation, a single core executing one thread's instructions one at a time is SISD at that level one instruction stream acting on one data stream. Many modern CPUs additionally support SIMD instruction-set extensions (e.g. SSE, AVX) within each core; confirming which specific extensions this machine supports would normally come from lscpu's Flags field in Task 1, so with Task 1 omitted this point is noted conceptually here rather than backed by measured evidence.

Discussion Questions

1. Why is a modern multicore CPU generally classified as MIMD at the machine level, even though each individual core may also execute SIMD instructions internally?

The machine-level classification describes how the cores relate to one another, not what happens inside any single core. Because each core fetches and executes its own independent instruction stream different threads can run entirely different code on different data simultaneously, as demonstrated in Task 2 the machine satisfies "multiple instruction, multiple data" at that level. A core's internal SIMD unit is a separate, lower-level detail that does not change how the cores as a whole relate to one another.

2. Explain, in your own words, why MISD is rarely seen in general-purpose computing and where it is occasionally used.

MISD requires multiple independent instruction streams to all operate on the same single data stream, which is rarely useful for ordinary computation since it does not parallelize independent work the way general-purpose workloads need. It appears mainly in fault-tolerant or redundant systems, such as space-shuttle flight computers, where several different implementations process the identical input independently so their outputs can be cross-checked for correctness or hardware faults.

3. Relate your Task 2 program to the MIMD classification: how does spawning one independent C# thread per core demonstrate MIMD behaviour in practice?

Each thread created in Task 2 runs the same Worker method as its own independently scheduled instruction stream, executing on its own logical core with its own private data its captured index and its own snapshot from Thread.GetCurrentProcessorId(). Because the OS scheduler runs

these threads concurrently on separate cores, each fetching and executing instructions independently on independent data, the program is a direct, working example of MIMD execution rather than a simulation of it.

4. If the lab machine's cores support AVX/SSE/NEON, give one concrete example of a computation that could exploit that SIMD capability, and briefly explain why.

Element-wise addition of two large numeric arrays, such as summing two vectors of doubles, is a natural fit: the same add operation is applied independently to every pair of elements, so a single SIMD instruction can add several elements at once within one core rather than looping one addition at a time. This matches SIMD's "single instruction, multiple data" pattern exactly, whereas Task 2's thread-per-core design instead exploits MIMD parallelism across cores the two techniques are complementary rather than competing.

Section 4: Task 4 - Shared-Memory vs. Distributed-Memory Implications

On the lab machine used in Tasks 2 and 3, all cores and all threads created in Task 2 share a single physical address space, communicating implicitly through shared variables and coordinating access using synchronization primitives such as lock or Monitor rather than by sending messages. A hypothetical cluster built from several machines identical to this one would instead give each node its own private physical memory, with processes on different nodes communicating explicitly over a network (e.g. sockets, gRPC, or a message queue), at a latency far higher than local shared-memory access.

Comparison Table

Aspect	Shared-Memory (This Machine)	Distributed-Memory (Hypothetical Cluster)
Address space	Single physical address space, shared by all threads	Private per node; no node can directly read another's memory
Typical communication mechanism	Reads/writes to shared variables	Explicit message passing over the network (sockets, gRPC, MPI)
Approximate communication latency	Nanoseconds (cache/memory access)	Microseconds to milliseconds (network round-trip)
Synchronization primitives used	lock / Monitor / Interlocked / ManualResetEvent	Send/receive, barriers, collective operations
Maximum practical scale	Bounded by cores in one machine (Task 2's core count)	Scales by adding more nodes
Effect of a single-unit failure	An unhandled exception in one thread can crash the whole process	A failed node can be isolated; other nodes keep running, but its in-flight work must be detected and handled

Programming model / API used here	System.Threading.Thread (Task 2)	Message-passing framework, e.g. MPI or gRPC (not used in this lab)
--------------------------------------	----------------------------------	---

Discussion Questions

1. If the workload from Task 2 were scaled up to run across a 4-node cluster of identical machines, what new source of overhead would appear that was not present when all threads ran on a single machine?

Network communication overhead would appear: coordinating work across nodes requires explicit message passing distributing work, collecting results, and synchronizing progress over a network link, introducing latency and serialization cost that simply did not exist when every thread shared one address space on a single machine.

2. Explain why NUMA can be viewed as an intermediate case between pure shared-memory and pure distributed-memory.

A remote NUMA-node access still goes through a single shared, globally addressable memory space logically identical to a local access, unlike a distributed system where remote memory is not addressable at all but it costs more latency than a local access, similar in spirit to a network hop. What is fundamentally different from a real network message is that no explicit send/receive call or message-passing code is needed: the hardware and OS handle the remote access transparently through ordinary load/store instructions, just at a higher latency than local memory.

3. For a computation with very frequent, fine-grained communication between subtasks, would you expect this machine's shared-memory model or a distributed-memory cluster to perform better? Justify your answer using the latency figures from the comparison table.

The shared-memory model would perform far better. As shown in the comparison table, shared-memory access costs nanoseconds versus microseconds-to-milliseconds for a network message; at fine granularity, the communication itself would dominate runtime on a cluster, since every iteration would pay a network round-trip that is roughly three to six orders of magnitude slower than a local memory access.

4. For a computation that can be split into large, mostly-independent chunks of work with little communication, would a distributed-memory cluster of several such machines be a reasonable way to scale beyond what this single machine can do? Explain why.

Yes. When communication is infrequent relative to the amount of independent work per chunk for example, processing millions of independent records the network latency cost is amortized over a large amount of computation, and the cluster's ability to add more nodes lets total throughput scale well beyond what a single machine's fixed core count could ever provide.